Report - Version 11: Subword Tokenization and Public Dataset

Introduction

Version 11 keeps the small decoder-only Transformer introduced in Version 10 and changes how language enters the model. The two reusable causal Transformer blocks, residual connections, LayerNorm operations, feed-forward networks and next-token objective are preserved.

The tiny hardcoded character corpus is replaced by the public WikiText-103 Raw v1 dataset, and character identifiers are replaced by Byte Pair Encoding (BPE) subword token identifiers. The original training, validation and test split boundaries are preserved, while controlled subsets keep the experiment small enough to remain inspectable and reproducible.

```
WikiText-103 text
  |
BPE tokenizer trained on the selected training subset
  |
2048-token vocabulary
  |
16-token context + 16 sequential targets
  |
Token + positional embeddings
  |
Transformer block 1
  |
Transformer block 2
  |
Vocabulary logits at every position
  |
Next-token prediction and greedy generation
```

Goal of Version 11

- replace the tiny hardcoded character corpus with a documented public language-modeling dataset;
- replace character-level tokenization with a BPE subword tokenizer trained only on the selected training subset;
- preserve the official WikiText training, validation and test split boundaries;
- construct fixed-length token sequences for next-token prediction without crossing text-row boundaries;
- keep the same two-block decoder-only Transformer architecture developed in Version 10;
- evaluate training, validation and test losses separately and inspect BPE predictions, generation and causal attention;
- keep the implementation explicit and computationally small while introducing public-data subword language modeling.

Architecture

The complete Version 11 flow is:

```
Dataset: WikiText-103 Raw v1
  |
Original rows: train 1,801,350 / validation 3,760 / test 4,358
  |
Selected rows: train 10,000 / validation 1,000 / test 1,000
  |
BPE vocabulary: 2048 tokens
  |
Raw fixed-length windows: 96,893 / 9,764 / 9,572
  |
Used sequences: train 12,000 / validation 2,000 / test 2,000
  |
Input tensor:  (N, 16)
Target tensor: (N, 16)
  |
Token embeddings: (2048, 8) + positions: (16, 8)
  |
Transformer block 1: (16, 8) -> (16, 8)
  |
Transformer block 2: (16, 8) -> (16, 8)
  |
Vocabulary logits: (16, 2048) per example
  |
Mini-batch gradient descent + separate validation/test evaluation
  |
Greedy autoregressive BPE generation
```

Public dataset and controlled subsets

The notebook reads the original WikiText-103 Raw v1 Parquet files: two training shards, one validation file and one test file. Empty text rows are removed before a seed-controlled subset is sampled independently from each official split. No new random train/validation split is created.

The dataset language is English. Source, provenance and licensing information are documented in the attached Kaggle dataset metadata used by the notebook.

BPE tokenizer and token sequences

A Byte Pair Encoding tokenizer is trained only on the selected training subset. The tokenizer uses a maximum vocabulary size of 2048, a special [UNK] token and an </w> suffix to mark the end of a pre-tokenized word. Validation and test text do not participate in vocabulary construction.

Each WikiText row is encoded independently. Windows contain CONTEXT_LENGTH + 1 = 17 tokens: the first 16 tokens form the input and the same sequence shifted by one position forms the 16-token target. A stride of 16 avoids excessive overlap, incomplete final windows are discarded, and examples never cross row boundaries.

Trainable parameters

The model contains 23 trainable TensorFlow variables and exactly 34,496 trainable parameters:

```
Token embeddings              2048 x 8    = 16384
Positional embeddings            16 x 8    =   128
Transformer block 1                         =   800
Transformer block 2                         =   800
Output weights                8 x 2048    = 16384
------------------------------------------------
Total                                       = 34496
```

Each Transformer block still contributes exactly 800 parameters:

```
Query projection               8 x 16 = 128
Key projection                 8 x 16 = 128
Value projection               8 x 16 = 128
Attention output projection   16 x 8  = 128
LayerNorm 1 scale + shift      8 + 8   =  16
Feed-forward input             8 x 16 = 128
Feed-forward output           16 x 8  = 128
LayerNorm 2 scale + shift      8 + 8   =  16
-----------------------------------------
Per block                                 = 800
```

Transformer block details

The internal Transformer mathematics is preserved from Version 10. For an input representation X, each block computes independent query, key and value projections, applies causal scaled dot-product attention, projects the attention result back to the 8-dimensional model representation, and then applies residual connections, LayerNorm and a position-wise 8 -> 16 -> 8 feed-forward network.

```
Q = X Wq
K = X Wk
V = X Wv

scores = Q K^T / sqrt(dk)
masked_scores = causal_mask(scores)
A = softmax(masked_scores)
H = A V
```

No bias terms are used in the attention, feed-forward or vocabulary projections. Both LayerNorm operations retain trainable scale and shift parameters.

Causal mask

```
[[1, 0, 0, ..., 0],
 [1, 1, 0, ..., 0],
 [1, 1, 1, ..., 0],
 ...
 [1, 1, 1, ..., 1]]
```

Loss, training and reproducibility

Sparse softmax cross-entropy is computed at all 16 sequence positions. Each mini-batch loss is the mean over its token targets, and dataset-level losses are accumulated with token weighting so the final partial batch is handled correctly.

tf.GradientTape computes gradients for all 23 trainable variables. Token embedding lookups can produce sparse IndexedSlices gradients; these are updated directly with scatter_sub, while dense gradients use assign_sub.

```
Learning rate: 1.0
Batch size:    32
Epochs:        20
Seed:          42

Training sequences:   12,000  -> 375 batches
Validation sequences:  2,000  ->  63 batches
Test sequences:        2,000  ->  63 batches
```

Saved training result

```
Initial train loss:        7.626767
Final train loss:          4.987922
Initial validation loss:   7.626187
Final validation loss:     5.085637
Test loss:                 5.117153
Uniform-vocabulary baseline: 7.624619
```

In the saved execution, training and validation losses decrease well below the uniform-vocabulary baseline, and the independent selected test subset produces a loss close to validation. These values describe this saved run; small numerical differences may appear across TensorFlow environments.

Inspection of learned next-token probabilities

For the unseen test context "Little Gidding is the fourth and final poem", the highest final-position probabilities in the saved run are:

```
','      0.1392
'.'      0.0825
'and'    0.0471
'of'     0.0453
"'"      0.0336
'('      0.0253
'was'    0.0252
```

The true next token is "of</w>" with model probability 0.0453, placing it among the highest-probability candidates in this execution.

Inspection of stacked causal attention

The final-position attention of Block 1 is almost uniform across the 16 context tokens, with weights close to 0.0625. Block 2 is more selective; its largest saved weights include G = 0.2545, L = 0.1621, f = 0.0861 and the = 0.0600. The final-position attention sum is 1.000000 in both blocks, while the full tests confirm zero attention to future positions.

Autoregressive generation

Generation starts from a text prompt, encodes it with the same BPE tokenizer, keeps the most recent 16 token identifiers as context and repeatedly selects the highest-probability next token. Version 11 therefore uses deterministic greedy decoding rather than stochastic sampling.

The saved demonstration begins from:

```
Little Gidding is the fourth and final poem
```

and generates 40 additional BPE tokens. The decoded saved output is:

```
Little Gidding is the fourth and final poem , and the Sea , and the Sea , and the Sea
, and the Sea , and the Sea , and the Sea , and the S
```

The repeated phrase is consistent with the deliberately small model, 16-token context and greedy selection strategy. Because whitespace pre-tokenization treats punctuation as separate units, decoded text can also contain spaces before punctuation; this is a tokenizer-formatting effect rather than a Transformer error.

Included checks

- the tokenizer vocabulary size is exactly 2048;
- training, validation and test input/target tensors have the configured 16-token sequence length and capped split sizes;
- token identifiers remain inside the vocabulary and the first target sequence is the input shifted by one token;
- the model output has shape (1, 16, 8), logits have shape (1, 16, 2048), and both attention matrices have shape (16, 16);
- the model contains two independent Transformer block objects, 23 trainable variables and exactly 34,496 trainable parameters;
- every attention row sums to 1 and all attention weights toward future positions are zero;
- training and validation loss histories contain 20 finite values, and the final training and validation losses improve over their initial values and remain below the uniform-vocabulary baseline;
- test loss is finite and below the uniform-vocabulary baseline;
- the inspected next-token probability distribution is finite and sums to 1;
- autoregressive generation adds exactly 40 new BPE tokens.

The notebook finishes its test section with All checks passed.

What Version 11 teaches

Version 11 is the first stage in the project where the language model moves from a tiny character-level demonstration to a public-text subword pipeline while preserving the decoder-only Transformer developed in Version 10. The key conceptual progression is:

```
public WikiText text
        -> BPE tokenizer trained on training data
        -> subword token identifiers
        -> fixed 16-token contexts and shifted targets
        -> token + positional embeddings
        -> Transformer block 1
        -> Transformer block 2
        -> 2048-token vocabulary logits
        -> next-token probabilities
        -> greedy BPE generation
```

The important distinction from Version 10 is not a new attention equation. The Transformer core remains deliberately unchanged so the effect of the new data source and language representation can be isolated and inspected directly.

Limitations

- only controlled subsets of WikiText-103 are used, so the experiment does not attempt full-scale dataset training;
- the model remains intentionally tiny, with 8-dimensional representations, two Transformer blocks and one attention mechanism per block rather than multi-head attention;
- the context length is fixed at 16 BPE tokens;
- the whitespace-based BPE setup is intentionally simple and can introduce spaces before punctuation during decoding;
- greedy decoding is deterministic and can amplify repetitive generation patterns;
- training retains the final-epoch parameters and does not yet implement checkpoint saving, restoration or resume logic;
- there is no learning-rate schedule, dropout, mixed precision or other larger-scale training infrastructure;
- evaluation remains intentionally lightweight; controlled generation strategies and more systematic evaluation are left for later versions.

Conclusion

Version 11 successfully replaces the character-level input pipeline with a BPE tokenizer trained on a documented public dataset while preserving the two-block causal decoder-only Transformer from Version 10. Official split boundaries remain separate, subword sequences are created independently inside WikiText rows, and the model now predicts over a 2048-token vocabulary instead of 27 characters.

The model is still deliberately small, but the project now has the data representation and public-corpus pipeline needed for the next stage. Version 12 can build on this foundation by introducing a dedicated training pipeline and checkpoints without changing the core conceptual transition completed here.